

afcestDataCheck: Comprehensive Reference Manual

High-Frequency Data Quality Checks for Survey Research – Version 0.3.0

Cedric Bationo, African Center for Studies and Training (AfCEST)

2026-02-28

Contents

1	Introduction	6
1.1	Motivation	6
1.2	Scope	6
1.3	How to Read This Manual	6
2	Installation	7
2.1	From GitHub (recommended)	7
2.2	From source	7
2.3	Optional dependencies	7
3	Quick Start	8
3.1	Step 1: Create sample data	8
3.2	Step 2: Run individual checks	8
3.3	Step 3: Run all checks via configuration	9
3.4	Step 4: Export results	10
4	The <code>check_result</code> Object	11
4.1	Structure	11
4.2	Constructor and helpers	11
5	Function Reference by Category	12
5.1	Identification Checks	12
5.1.1	<code>check_duplicate_ids()</code>	12
5.1.2	<code>check_duplicate_fingerprint()</code>	12

5.1.3	<code>check_missing_ids()</code>	13
5.1.4	<code>check_id_format()</code>	13
5.1.5	<code>check_id_in_sample()</code>	14
5.2	Metadata Checks	14
5.2.1	<code>check_consent()</code>	14
5.2.2	<code>check_form_version()</code>	14
5.2.3	<code>check_interview_completed()</code>	15
5.2.4	<code>check_survey_tracking()</code>	15
5.2.5	<code>check_respondent_eligibility()</code>	16
5.3	Completeness Checks	16
5.3.1	<code>check_missing_by_variable()</code>	16
5.3.2	<code>check_missing_by_enumerator()</code>	17
5.3.3	<code>check_all_missing_variables()</code>	17
5.3.4	<code>check_skip_pattern()</code>	17
5.3.5	<code>check_roster_completeness()</code>	18
5.4	Outlier Checks	18
5.4.1	<code>check_outliers_iqr()</code>	18
5.4.2	<code>check_outliers_zscore()</code>	19
5.4.3	<code>check_outliers_mad()</code>	19
5.4.4	<code>check_hard_range()</code>	20
5.4.5	<code>check_negative_values()</code>	20
5.5	Enumerator Performance Checks	20
5.5.1	<code>check_enumerator_productivity()</code>	21
5.5.2	<code>check_enumerator_duration()</code>	21
5.5.3	<code>check_dk_rate()</code>	21
5.5.4	<code>check_enumerator_time_gap()</code>	22
5.5.5	<code>check_straightlining()</code>	22
5.6	Timing Checks	23
5.6.1	<code>check_survey_duration()</code>	23
5.6.2	<code>check_collection_window()</code>	23
5.6.3	<code>check_future_dates()</code>	24
5.6.4	<code>check_survey_end_before_start()</code>	24
5.6.5	<code>check_duration_by_hh_size()</code>	25
5.7	GPS Validation Checks	25

5.7.1	check_gps_boundary()	25
5.7.2	check_gps_centroid_distance()	26
5.7.3	check_gps_accuracy()	26
5.7.4	check_gps_duplicates()	26
5.7.5	check_gps_swap()	27
5.7.6	check_gps_null_island()	27
5.7.7	check_gps_clustering()	28
5.7.8	check_gps_altitude()	28
5.8	Fabrication Detection Checks	29
5.8.1	check_benford_first_digit()	29
5.8.2	check_benford_second_digit()	29
5.8.3	check_digit_preference()	30
5.8.4	check_icc_enumerator()	30
5.8.5	check_response_entropy()	30
5.8.6	check_duplicate_response_patterns()	31
5.8.7	check_variance_ratio()	31
5.9	Logic Consistency Checks	32
5.9.1	check_hh_composition()	32
5.9.2	check_income_expenditure()	32
5.9.3	check_age_date_consistency()	33
5.9.4	check_custom_logic()	33
5.10	Text Quality Checks	34
5.10.1	check_other_specify()	34
5.10.2	check_text_length()	34
5.10.3	check_text_duplicates()	35
5.10.4	check_name_format()	35
5.10.5	check_phone_format()	35
5.11	Back-Check Checks	36
5.11.1	check_backcheck_t1_match()	36
5.11.2	check_backcheck_t2_match()	36
5.11.3	check_backcheck_t3_match()	37
5.11.4	check_backcheck_coverage()	37
5.11.5	check_backcheck_by_enumerator()	38
5.12	Paradata Checks	38

5.12.1	<code>check_text_audit_duration()</code>	38
5.12.2	<code>check_text_audit_sequence()</code>	39
5.12.3	<code>check_speed_violations()</code>	39
5.12.4	<code>check_field_comments()</code>	40
5.12.5	<code>check_photo_attachment()</code>	40
5.12.6	<code>check_device_consistency()</code>	41
5.13	Correction Functions	41
5.13.1	<code>apply_corrections()</code>	41
5.13.2	<code>check_correction_log()</code>	42
5.13.3	<code>recode_other_specify()</code>	42
6	Configuration System	44
6.1	YAML Structure	44
6.2	Configuration Functions	45
7	Orchestrator: <code>run_all_checks()</code>	47
7.1	Signature	47
7.2	Parameters	47
7.3	Return Value	47
7.4	Methods	47
8	Platform Normalization	49
8.1	Supported Platforms	49
8.2	Key Functions	49
9	Export Functions	50
9.1	Excel Export	50
9.2	CSV Export	50
10	Integration with Shiny	51
11	Appendix: Complete Function Index	53

afcestDataCheck is an R package providing 65+ automated high-frequency data quality check (HFC) functions for survey research in international development. The package covers 13 categories of checks—from duplicate identification and GPS validation to fabrication detection and back-check reconciliation—and supports five major data collection platforms (KoBoToolbox, SurveyCTO, ODK Central, Survey Solutions, and CSPro). All checks return a standardized `check_result` object, enabling seamless orchestration via YAML-driven configuration files and one-command export to formatted Excel workbooks. This vignette serves as the authoritative reference manual for the package.

1 Introduction

1.1 Motivation

Field surveys are the backbone of evidence-based policy in international development. Yet data quality problems—duplicate records, fabricated responses, GPS anomalies, enumerator misconduct—frequently compromise survey integrity. Traditional quality assurance relies on ad-hoc Stata do-files or manual spot checks, neither of which scales to the volume and velocity of modern mobile data collection.

afcestDataCheck addresses this gap by providing a comprehensive, modular R toolkit for automated high-frequency checks (HFCs). The package is designed with three guiding principles:

1. **Completeness.** 65+ check functions organized into 13 thematic categories cover the full HFC lifecycle, from sample tracking through back-check reconciliation.
2. **Standardization.** Every check function returns a uniform `check_result` list, making it trivial to aggregate, filter, and export results programmatically.
3. **Configurability.** A YAML-driven configuration system lets survey managers define project-specific parameters once and run the entire suite with a single call to `run_all_checks()`.

1.2 Scope

The package targets development-sector surveys including, but not limited to, agricultural production surveys, household expenditure surveys, health facility assessments (SDI), and food security monitoring (CFSVA, EFSA). It integrates natively with KoBoToolbox, SurveyCTO, ODK Central, Survey Solutions, and CSPro through a platform-normalization layer.

1.3 How to Read This Manual

Sections 2–4 provide a quick-start guide. Sections 5.1–5.13 constitute the comprehensive function reference, organized by category. Sections 6–10 describe the configuration system, orchestrator, platform normalization, and export facilities. Section 12 contains the complete function index as a reference table.

2 Installation

2.1 From GitHub (recommended)

```
# install.packages("pak")  
pak::pak("afcest/afcestDataCheck")
```

2.2 From source

```
# Clone the repository, then:  
devtools::install("path/to/survey-data-quality")
```

2.3 Optional dependencies

Several check functions use optional packages. Install them as needed:

```
# Spatial checks (GPS boundary, clustering)  
install.packages("sf")  
  
# Excel export with conditional formatting  
install.packages("openxlsx2")  
  
# Platform API access  
install.packages(c("robotoolbox", "rsurveycto", "ruODK"))
```

3 Quick Start

This section demonstrates the core workflow: load data, run checks, inspect results, and export.

3.1 Step 1: Create sample data

```
library(afcestDataCheck)

# Simulated household survey data
set.seed(42)
n <- 200

survey <- data.frame(
  hh_id      = sprintf("HH-%04d", 1:n),
  enum_id    = sample(paste0("EN", 1:8), n, replace = TRUE),
  consent    = sample(c(1, 0), n, replace = TRUE, prob = c(0.95, 0.05)),
  date       = as.Date("2025-06-01") + sample(0:29, n, replace = TRUE),
  duration_min = rnorm(n, mean = 45, sd = 15),
  hh_size    = rpois(n, lambda = 5) + 1,
  income     = rlnorm(n, meanlog = 12, sdlog = 1),
  expenditure = rlnorm(n, meanlog = 11.5, sdlog = 1.2),
  lat        = runif(n, 12.2, 12.5),
  lon        = runif(n, -1.6, -1.3),
  gps_accuracy = rexp(n, rate = 0.1),
  form_version = sample(c("v2.1", "v2.0"), n, replace = TRUE, prob = c(0.9, 0.1)),
  stringsAsFactors = FALSE
)

# Inject quality issues for demonstration
survey$hh_id[5]      <- survey$hh_id[3]      # duplicate ID
survey$duration_min[10] <- 3                  # too short
survey$income[15]    <- 1e9                  # extreme outlier
survey$lat[20]       <- 0                    # null island
```

3.2 Step 2: Run individual checks

```
# Check for duplicate IDs
res_dup <- check_duplicate_ids(survey, id_col = "hh_id")
print(res_dup)
#> -- check_result: duplicate_hh_id ----
#> Severity : error
#> Flagged  : 2 observation(s)
#> IDs      : HH-0003, HH-0003
```



```
# Check survey duration
res_dur <- check_survey_duration(
  survey, id_col = "hh_id", duration_col = "duration_min",
  min_duration = 15, max_duration = 120
)
print(res_dur)

# Check GPS null island
res_gps <- check_gps_null_island(
  survey, id_col = "hh_id", lat_col = "lat", lon_col = "lon"
)
print(res_gps)
```

3.3 Step 3: Run all checks via configuration

```
# Write a minimal YAML config
config_text <- "
project:
  name: Quick Start Demo

variables:
  id: hh_id
  enumerator: enum_id
  date: date
  duration: duration_min
  gps_latitude: lat
  gps_longitude: lon
  consent: consent

checks:
  identification:
    duplicate_ids: true
    missing_ids: true
  timing:
    duration:
      min_minutes: 15
      max_minutes: 120
  gps:
    null_island: true
"

config <- read_config(textConnection(config_text))
report <- run_all_checks(survey, config = config, verbose = TRUE)
print(report)
summary(report)
```

3.4 Step 4: Export results

```
# Export to a formatted Excel workbook
export_to_excel(report$results, data = survey, id_col = "hh_id",
               path = "hfc_report.xlsx")

# Or export to CSV
export_to_csv(report$results, output_dir = "hfc_output/")
```

4 The check_result Object

Before diving into the function reference, it is essential to understand the data structure that every check function returns.

4.1 Structure

Every check function returns a named list of class `check_result` with the following fields:

Table 1: Fields of a `check_result` object

Field	Type	Description
<code>check_name</code>	<code>character(1)</code>	Machine-readable check identifier
<code>check_category</code>	<code>character(1)</code>	Category (e.g., “identification”, “enumerator”)
<code>n_flagged</code>	<code>integer(1)</code>	Number of observations flagged
<code>n_total</code>	<code>integer(1)</code>	Total number of records examined
<code>flagged_ids</code>	<code>character()</code>	Vector of IDs that were flagged
<code>flag_reason</code>	<code>character()</code>	Human-readable reason per flag
<code>severity</code>	<code>character(1)</code>	One of "error", "warning", "info"
<code>summary_stat</code>	<code>list()</code>	Check-specific summary statistics
<code>timestamp</code>	<code>POSIXct</code>	Timestamp when the check was run

4.2 Constructor and helpers

```
# Create a check_result manually (used internally)
res <- new_check_result(
  check_name      = "my_custom_check",
  check_category  = "logic",
  n_flagged       = 3L,
  n_total         = 200L,
  flagged_ids     = c("HH-001", "HH-007", "HH-042"),
  flag_reason     = rep("Custom logic violation", 3),
  severity        = "warning",
  summary_stat    = list(total_obs = 200, pct_flagged = 1.5)
)

# Validate
is_check_result(res)
#> [1] TRUE

# Pretty print
print(res)

# Combine multiple results
combined <- bind_check_results(list(res_dup, res_dur, res_gps))
```

5 Function Reference by Category

This section documents every check function in the package, organized by thematic category. For each function we provide: the full signature with default arguments, a brief description, a code example, and notes on the returned `check_result`.

5.1 Identification Checks

These checks ensure that survey records can be uniquely and correctly identified, and that the sample frame is respected.

5.1.1 `check_duplicate_ids()`

Signature:

```
check_duplicate_ids(data, id_col)
```

Description: Identifies rows with non-unique values in `id_col`. All rows sharing a duplicated ID are flagged. Severity: **error**.

Example:

```
res <- check_duplicate_ids(survey, id_col = "hh_id")
res$n_flagged
#> [1] 2
res$flagged_ids
#> [1] "HH-0003" "HH-0003"
```

Returns: `check_result` with `check_name` = "duplicate_hh_id". `summary_stat` includes the count of unique duplicated values.

5.1.2 `check_duplicate_fingerprint()`

Signature:

```
check_duplicate_fingerprint(data, id_col, quasi_ids)
```

Description: Detects near-duplicates by comparing combinations of quasi-identifier columns (e.g., name, date of birth, village). Even if `id_col` values differ, records sharing identical quasi-identifiers are flagged. Severity: **warning**.

Example:

```
res <- check_duplicate_fingerprint(  
  survey, id_col = "hh_id",  
  quasi_ids = c("head_name", "village", "hh_size")  
)
```

Returns: check_result with check_name = "duplicate_fingerprint".

5.1.3 check_missing_ids()

Signature:

```
check_missing_ids(data, id_col)
```

Description: Flags observations where id_col is NA, empty string, or whitespace-only. Severity: error.

Example:

```
res <- check_missing_ids(survey, id_col = "hh_id")  
res$n_flagged  
#> [1] 0
```

5.1.4 check_id_format()

Signature:

```
check_id_format(data, id_col, pattern)
```

Description: Validates that every ID matches the supplied regular expression pattern. Useful for enforcing project-specific ID conventions (e.g., \texttt{"^HH-[0-9]{4}\$"}). Severity: warning.

Example:

```
res <- check_id_format(  
  survey, id_col = "hh_id", pattern = "^HH-[0-9]{4}$"  
)
```

5.1.5 check_id_in_sample()

Signature:

```
check_id_in_sample(data, id_col, sampling_frame)
```

Description: Cross-references collected IDs against the pre-defined sampling frame. IDs not found in the frame are flagged as potential replacements or unauthorized additions. Severity: **warning**.

Example:

```
frame <- data.frame(hh_id = sprintf("HH-%04d", 1:250))
res <- check_id_in_sample(survey, id_col = "hh_id", sampling_frame = frame)
```

5.2 Metadata Checks

These checks validate survey-level metadata such as consent status, form versions, completion flags, and respondent eligibility.

5.2.1 check_consent()

Signature:

```
check_consent(data, id_col, consent_col, consent_value = 1)
```

Description: Flags observations where the consent field does not equal `consent_value`. Records without consent should typically be excluded from analysis. Severity: **error**.

Example:

```
res <- check_consent(
  survey, id_col = "hh_id",
  consent_col = "consent", consent_value = 1
)
```

5.2.2 check_form_version()

Signature:

```
check_form_version(data, id_col, version_col, expected_version)
```

Description: Identifies submissions collected with an outdated or incorrect form version. Severity: warning.

Example:

```
res <- check_form_version(  
  survey, id_col = "hh_id",  
  version_col = "form_version", expected_version = "v2.1"  
)
```

5.2.3 check_interview_completed()

Signature:

```
check_interview_completed(data, id_col, status_col,  
  complete_value = "complete")
```

Description: Flags interviews that were not marked as fully completed. Severity: warning.

Example:

```
res <- check_interview_completed(  
  survey, id_col = "hh_id",  
  status_col = "status", complete_value = "complete"  
)
```

5.2.4 check_survey_tracking()

Signature:

```
check_survey_tracking(data, id_col, stratum_col, target_per_stratum)
```

Description: Compares the number of completed surveys per stratum against the sampling targets. Reports under- and over-sampling by stratum. Severity: info.

Example:

```
targets <- c(region_A = 50, region_B = 75, region_C = 75)
res <- check_survey_tracking(
  survey, id_col = "hh_id",
  stratum_col = "region", target_per_stratum = targets
)
```

Returns: summary_stat includes per-stratum counts, targets, and deviations.

5.2.5 check_respondent_eligibility()

Signature:

```
check_respondent_eligibility(data, id_col, age_col = NULL,
                             gender_col = NULL, min_age = 18,
                             max_age = NULL, eligible_gender = NULL)
```

Description: Validates that respondents meet age and/or gender eligibility criteria. Severity: warning.

Example:

```
res <- check_respondent_eligibility(
  survey, id_col = "hh_id",
  age_col = "resp_age", min_age = 18, max_age = 65
)
```

5.3 Completeness Checks

These functions assess the extent and pattern of missing data at the variable, enumerator, and observation levels.

5.3.1 check_missing_by_variable()

Signature:

```
check_missing_by_variable(data, id_col, threshold = 0.05)
```

Description: Computes the proportion of missing values for each column. Variables exceeding threshold are flagged. Severity: warning.

Example:


```
res <- check_missing_by_variable(survey, id_col = "hh_id", threshold = 0.05)
res$summary_stat
# A list of variable names and their missingness rates
```

5.3.2 check_missing_by_enumerator()

Signature:

```
check_missing_by_enumerator(data, id_col, enum_col, threshold_sd = 2)
```

Description: For each enumerator, computes overall missingness rates and flags those whose rate exceeds the group mean by more than `threshold_sd` standard deviations. Severity: **warning**.

Example:

```
res <- check_missing_by_enumerator(
  survey, id_col = "hh_id", enum_col = "enum_id", threshold_sd = 2
)
```

5.3.3 check_all_missing_variables()

Signature:

```
check_all_missing_variables(data)
```

Description: Identifies columns where every value is NA. These may indicate programming errors in the survey instrument. Severity: **error**.

Example:

```
res <- check_all_missing_variables(survey)
```

5.3.4 check_skip_pattern()

Signature:

```
check_skip_pattern(data, id_col, parent_col, child_col,  
  parent_value, expect_child_na = TRUE)
```

Description: Validates skip logic: when `parent_col` equals `parent_value`, the `child_col` should be NA (if `expect_child_na = TRUE`) or non-missing (if `FALSE`). Severity: `warning`.

Example:

```
res <- check_skip_pattern(  
  survey, id_col = "hh_id",  
  parent_col = "has_livestock", child_col = "n_cattle",  
  parent_value = 0, expect_child_na = TRUE  
)
```

5.3.5 check_roster_completeness()

Signature:

```
check_roster_completeness(data, id_col, hh_size_col, member_count_col)
```

Description: Flags households where the reported household size does not match the number of roster entries. Severity: `warning`. Also used in logic consistency checks (Section 5.9).

Example:

```
res <- check_roster_completeness(  
  survey, id_col = "hh_id",  
  hh_size_col = "hh_size", member_count_col = "n_members"  
)
```

5.4 Outlier Checks

These functions detect extreme or implausible numeric values using statistical and rule-based methods.

5.4.1 check_outliers_iqr()

Signature:

```
check_outliers_iqr(data, id_col, num_col, multiplier = 1.5)
```

Description: Flags values outside the interquartile range fence: $[Q_1 - k \cdot \text{IQR}, Q_3 + k \cdot \text{IQR}]$ where $k = \text{multiplier}$. Severity: **warning**.

Example:

```
res <- check_outliers_iqr(  
  survey, id_col = "hh_id", num_col = "income", multiplier = 1.5  
)
```

Returns: `summary_stat` includes Q1, Q3, IQR, `lower_fence`, and `upper_fence`.

5.4.2 check_outliers_zscore()

Signature:

```
check_outliers_zscore(data, id_col, num_col, threshold = 3)
```

Description: Flags values with $|z| > \text{threshold}$, where $z = (x - \bar{x})/s$. Sensitive to non-normal distributions; consider `check_outliers_mad()` for skewed data. Severity: **warning**.

Example:

```
res <- check_outliers_zscore(  
  survey, id_col = "hh_id", num_col = "income", threshold = 3  
)
```

5.4.3 check_outliers_mad()

Signature:

```
check_outliers_mad(data, id_col, num_col, threshold = 3)
```

Description: Uses the Median Absolute Deviation (MAD) as a robust alternative to the z-score. Flags values where $|x - \tilde{x}|/\text{MAD} > \text{threshold}$. Severity: **warning**.

Example:

```
res <- check_outliers_mad(  
  survey, id_col = "hh_id", num_col = "income", threshold = 3  
)
```

5.4.4 check_hard_range()

Signature:

```
check_hard_range(data, id_col, num_col, min_val, max_val)
```

Description: Applies hard-coded minimum and maximum bounds. Values outside `[min_val, max_val]` are flagged. Severity: **error**.

Example:

```
res <- check_hard_range(  
  survey, id_col = "hh_id", num_col = "hh_size",  
  min_val = 1, max_val = 30  
)
```

5.4.5 check_negative_values()

Signature:

```
check_negative_values(data, id_col, num_col)
```

Description: Flags any negative values in a column that should contain non-negative quantities (e.g., income, area, count). Severity: **error**.

Example:

```
res <- check_negative_values(survey, id_col = "hh_id", num_col = "income")
```

5.5 Enumerator Performance Checks

These functions monitor enumerator behavior to detect low-quality or potentially fabricated data collection patterns.

5.5.1 check_enumerator_productivity()

Signature:

```
check_enumerator_productivity(data, id_col, enum_col, date_col,  
                              max_daily = 10)
```

Description: Counts the number of interviews per enumerator per day. Flags enumerator-day combinations exceeding `max_daily`. Severity: warning.

Example:

```
res <- check_enumerator_productivity(  
  survey, id_col = "hh_id", enum_col = "enum_id",  
  date_col = "date", max_daily = 10  
)
```

5.5.2 check_enumerator_duration()

Signature:

```
check_enumerator_duration(data, id_col, enum_col, duration_col,  
                           threshold_sd = 2)
```

Description: Computes mean interview duration per enumerator and flags those whose average deviates by more than `threshold_sd` standard deviations from the overall mean. Severity: warning.

Example:

```
res <- check_enumerator_duration(  
  survey, id_col = "hh_id", enum_col = "enum_id",  
  duration_col = "duration_min", threshold_sd = 2  
)
```

5.5.3 check_dk_rate()

Signature:

```
check_dk_rate(data, id_col, enum_col,
              dk_value = c("dk", "don't know", "-99"),
              threshold_sd = 2)
```

Description: Calculates the rate of “don’t know” or refusal responses per enumerator across all variables. Enumerators with unusually high DK rates may require retraining. Severity: **warning**.

Example:

```
res <- check_dk_rate(
  survey, id_col = "hh_id", enum_col = "enum_id",
  dk_value = c("dk", "don't know", "-99", "98"),
  threshold_sd = 2
)
```

5.5.4 check_enumerator_time_gap()

Signature:

```
check_enumerator_time_gap(data, id_col, enum_col, time_col,
                           min_gap_minutes = 5)
```

Description: Flags consecutive interviews by the same enumerator with a time gap shorter than `min_gap_minutes`. An impossibly short gap may indicate fabrication. Severity: **warning**.

Example:

```
res <- check_enumerator_time_gap(
  survey, id_col = "hh_id", enum_col = "enum_id",
  time_col = "start_time", min_gap_minutes = 5
)
```

5.5.5 check_straightlining()

Signature:

```
check_straightlining(data, id_col, check_cols,
                      max_identical_rate = 0.8)
```

Description: For each observation, computes the proportion of identical responses across the specified columns. Observations exceeding `max_identical_rate` are flagged as potential straightliners. Severity: `warning`. Also referenced in fabrication detection (Section 5.8).

Example:

```
likert_cols <- paste0("q_", 1:15)
res <- check_straightlining(
  survey, id_col = "hh_id",
  check_cols = likert_cols, max_identical_rate = 0.8
)
```

5.6 Timing Checks

These functions validate temporal aspects of data collection, including interview duration, date validity, and collection windows.

5.6.1 `check_survey_duration()`

Signature:

```
check_survey_duration(data, id_col, duration_col,
  min_duration = 15, max_duration = 120)
```

Description: Flags interviews whose duration falls outside `[min_duration,max_duration]` minutes. Severity: `warning`.

Example:

```
res <- check_survey_duration(
  survey, id_col = "hh_id", duration_col = "duration_min",
  min_duration = 15, max_duration = 120
)
```

5.6.2 `check_collection_window()`

Signature:

```
check_collection_window(data, id_col, date_col,
  start_date, end_date)
```

Description: Flags interviews with dates outside the official data collection window. Severity: warning.

Example:

```
res <- check_collection_window(  
  survey, id_col = "hh_id", date_col = "date",  
  start_date = as.Date("2025-06-01"),  
  end_date   = as.Date("2025-06-30")  
)
```

5.6.3 check_future_dates()

Signature:

```
check_future_dates(data, id_col, date_col,  
  reference_date = Sys.Date())
```

Description: Flags dates that are in the future relative to `reference_date`. Severity: error.

Example:

```
res <- check_future_dates(  
  survey, id_col = "hh_id", date_col = "date"  
)
```

5.6.4 check_survey_end_before_start()

Signature:

```
check_survey_end_before_start(data, id_col, start_col, end_col)
```

Description: Flags records where the end timestamp precedes the start timestamp. Severity: error. Also referenced in logic consistency checks (Section 5.9).

Example:

```
res <- check_survey_end_before_start(  
  survey, id_col = "hh_id",  
  start_col = "start_time", end_col = "end_time"  
)
```

5.6.5 check_duration_by_hh_size()

Signature:

```
check_duration_by_hh_size(data, id_col, duration_col,
                          hh_size_col,
                          min_minutes_per_member = 3)
```

Description: Flags interviews where the duration is implausibly low given household size. The minimum expected duration is computed as $hh_size \times min_minutes_per_member$. Severity: warning. Also referenced in logic consistency checks (Section 5.9).

Example:

```
res <- check_duration_by_hh_size(
  survey, id_col = "hh_id",
  duration_col = "duration_min", hh_size_col = "hh_size",
  min_minutes_per_member = 3
)
```

5.7 GPS Validation Checks

These functions validate geographic coordinates for boundary compliance, accuracy, duplication, and common GPS artifacts.

5.7.1 check_gps_boundary()

Signature:

```
check_gps_boundary(data, id_col, lat_col, lon_col,
                   boundary_bbox = NULL, boundary_sf = NULL)
```

Description: Flags GPS coordinates that fall outside the study area boundary. The boundary can be specified as a bounding box (named numeric vector: `xmin`, `ymin`, `xmax`, `ymax`) or an `sf` polygon object. Severity: error.

Example:

```
bbox <- c(xmin = -1.7, ymin = 12.1, xmax = -1.2, ymax = 12.6)
res <- check_gps_boundary(
  survey, id_col = "hh_id",
  lat_col = "lat", lon_col = "lon",
  boundary_bbox = bbox
)
```

5.7.2 check_gps_centroid_distance()

Signature:

```
check_gps_centroid_distance(data, id_col, lat_col, lon_col,  
                           cluster_col, max_distance_km = 10)
```

Description: Computes the distance of each point from its cluster centroid and flags points exceeding `max_distance_km`. Severity: `warning`.

Example:

```
res <- check_gps_centroid_distance(  
  survey, id_col = "hh_id",  
  lat_col = "lat", lon_col = "lon",  
  cluster_col = "village", max_distance_km = 10  
)
```

5.7.3 check_gps_accuracy()

Signature:

```
check_gps_accuracy(data, id_col, accuracy_col, max_accuracy = 50)
```

Description: Flags GPS readings with an accuracy value (in meters) exceeding `max_accuracy`. High accuracy values indicate poor satellite reception. Severity: `warning`.

Example:

```
res <- check_gps_accuracy(  
  survey, id_col = "hh_id",  
  accuracy_col = "gps_accuracy", max_accuracy = 50  
)
```

5.7.4 check_gps_duplicates()

Signature:

```
check_gps_duplicates(data, id_col, lat_col, lon_col,  
                    min_distance = 0.0001)
```

Description: Identifies pairs of records with GPS coordinates closer than `min_distance` degrees. Duplicate GPS points may indicate that an enumerator did not actually travel to the household. Severity: **warning**.

Example:

```
res <- check_gps_duplicates(  
  survey, id_col = "hh_id",  
  lat_col = "lat", lon_col = "lon", min_distance = 0.0001  
)
```

5.7.5 check_gps_swap()

Signature:

```
check_gps_swap(data, id_col, lat_col, lon_col, expected_bbox)
```

Description: Detects cases where latitude and longitude appear to be swapped, by checking if swapping the values brings the point inside the `expected_bbox`. Severity: **error**.

Example:

```
bbox <- c(xmin = -1.7, ymin = 12.1, xmax = -1.2, ymax = 12.6)  
res <- check_gps_swap(  
  survey, id_col = "hh_id",  
  lat_col = "lat", lon_col = "lon", expected_bbox = bbox  
)
```

5.7.6 check_gps_null_island()

Signature:

```
check_gps_null_island(data, id_col, lat_col, lon_col,  
  null_values = c(0, 999, -999))
```

Description: Flags records where latitude or longitude equal sentinel null values (0, 999, -999, etc.), indicating missing or default GPS readings. The coordinates (0, 0) are known as “Null Island.” Severity: **error**.

Example:

```
res <- check_gps_null_island(  
  survey, id_col = "hh_id",  
  lat_col = "lat", lon_col = "lon",  
  null_values = c(0, 999, -999)  
)
```

5.7.7 check_gps_clustering()

Signature:

```
check_gps_clustering(data, id_col, lat_col, lon_col,  
  enum_col, max_same_point = 3)
```

Description: Flags enumerators who report more than `max_same_point` interviews at identical GPS coordinates. Severity: warning.

Example:

```
res <- check_gps_clustering(  
  survey, id_col = "hh_id",  
  lat_col = "lat", lon_col = "lon",  
  enum_col = "enum_id", max_same_point = 3  
)
```

5.7.8 check_gps_altitude()

Signature:

```
check_gps_altitude(data, id_col, alt_col,  
  min_alt = -500, max_alt = 6000)
```

Description: Flags implausible altitude readings outside `[min_alt,max_alt]` meters. Severity: warning.

Example:

```
res <- check_gps_altitude(  
  survey, id_col = "hh_id",  
  alt_col = "altitude", min_alt = 0, max_alt = 2000  
)
```

5.8 Fabrication Detection Checks

These advanced statistical checks are designed to detect fabricated or manipulated survey data. They rely on mathematical properties (Benford's Law, entropy, ICC) that genuine data typically exhibits.

5.8.1 `check_benford_first_digit()`

Signature:

```
check_benford_first_digit(data, id_col, num_col)
```

Description: Tests the distribution of first (leading) digits in `num_col` against Benford's Law using a chi-squared goodness-of-fit test. Significant deviation may suggest data fabrication. Severity: `info`.

Example:

```
res <- check_benford_first_digit(  
  survey, id_col = "hh_id", num_col = "income"  
)  
res$summary_stat$p_value
```

Returns: `summary_stat` includes the chi-squared statistic, p-value, and observed vs. expected digit frequencies.

5.8.2 `check_benford_second_digit()`

Signature:

```
check_benford_second_digit(data, id_col, num_col)
```

Description: Same as above, but tests the distribution of second digits. Severity: `info`.

Example:

```
res <- check_benford_second_digit(  
  survey, id_col = "hh_id", num_col = "income"  
)
```

5.8.3 check_digit_preference()

Signature:

```
check_digit_preference(data, id_col, num_col)
```

Description: Detects heaping or rounding bias by analyzing the distribution of terminal digits. Excessive clustering on 0 or 5 suggests that respondents or enumerators are rounding values. Severity: **info**.

Example:

```
res <- check_digit_preference(  
  survey, id_col = "hh_id", num_col = "income"  
)
```

5.8.4 check_icc_enumerator()

Signature:

```
check_icc_enumerator(data, id_col, enum_col, num_cols,  
                      max_icc = 0.15)
```

Description: Computes the Intra-class Correlation Coefficient (ICC) for numeric variables, treating enumerator as the grouping factor. A high ICC indicates that more variance is explained by enumerator identity than by actual respondent differences, which is a hallmark of fabrication. Severity: **warning**.

Example:

```
res <- check_icc_enumerator(  
  survey, id_col = "hh_id", enum_col = "enum_id",  
  num_cols = c("income", "expenditure", "hh_size"),  
  max_icc = 0.15  
)
```

5.8.5 check_response_entropy()

Signature:

```
check_response_entropy(data, id_col, check_cols,  
                        min_entropy_ratio = 0.3)
```

Description: Computes the normalized Shannon entropy for each observation across `check_cols`. Low entropy indicates little variation in responses, suggesting straightlining or fabrication. Severity: warning.

Example:

```
res <- check_response_entropy(  
  survey, id_col = "hh_id",  
  check_cols = paste0("q_", 1:20),  
  min_entropy_ratio = 0.3  
)
```

5.8.6 check_duplicate_response_patterns()

Signature:

```
check_duplicate_response_patterns(data, id_col, check_cols,  
                                  similarity_threshold = 0.95)
```

Description: Computes pairwise similarity (proportion of identical responses) across observations. Pairs exceeding `similarity_threshold` are flagged. Severity: warning.

Example:

```
res <- check_duplicate_response_patterns(  
  survey, id_col = "hh_id",  
  check_cols = paste0("q_", 1:20),  
  similarity_threshold = 0.95  
)
```

5.8.7 check_variance_ratio()

Signature:

```
check_variance_ratio(data, id_col, enum_col, num_cols,  
                     max_ratio = 2)
```

Description: For each numeric variable, computes the ratio of between-enumerator variance to within-enumerator variance. Enumerators with ratios exceeding `max_ratio` may be producing artificially homogeneous data. Severity: **warning**.

Example:

```
res <- check_variance_ratio(  
  survey, id_col = "hh_id", enum_col = "enum_id",  
  num_cols = c("income", "expenditure"), max_ratio = 2  
)
```

5.9 Logic Consistency Checks

These functions validate internal consistency between related survey variables.

5.9.1 check_hh_composition()

Signature:

```
check_hh_composition(data, id_col, hh_size_col, roster_count_col)
```

Description: Flags households where the reported total household size does not match the count of roster members. Severity: **warning**.

Example:

```
res <- check_hh_composition(  
  survey, id_col = "hh_id",  
  hh_size_col = "hh_size", roster_count_col = "n_members"  
)
```

5.9.2 check_income_expenditure()

Signature:

```
check_income_expenditure(data, id_col, income_col,  
                          expenditure_col, max_ratio = 5)
```

Description: Flags households where the ratio of income to expenditure (or vice versa) exceeds `max_ratio`. Extreme ratios may indicate data entry errors. Severity: **warning**.

Example:


```
res <- check_income_expenditure(  
  survey, id_col = "hh_id",  
  income_col = "income", expenditure_col = "expenditure",  
  max_ratio = 5  
)
```

5.9.3 check_age_date_consistency()

Signature:

```
check_age_date_consistency(data, id_col, age_col, dob_col,  
  tolerance_years = 1)
```

Description: Compares reported age against computed age from date of birth. Discrepancies exceeding `tolerance_years` are flagged. Severity: **warning**.

Example:

```
res <- check_age_date_consistency(  
  survey, id_col = "hh_id",  
  age_col = "resp_age", dob_col = "resp_dob",  
  tolerance_years = 1  
)
```

5.9.4 check_custom_logic()

Signature:

```
check_custom_logic(data, id_col, condition_expr, description)
```

Description: Evaluates an arbitrary R expression (`condition_expr`) against the data and flags rows where the condition is TRUE. This is the escape hatch for project-specific logic checks not covered by built-in functions. Severity: **warning**.

Example:

```
res <- check_custom_logic(  
  survey, id_col = "hh_id",  
  condition_expr = quote(hh_size > 15 & income < 50000),  
  description = "Large household with very low income"  
)
```

Note: The functions `check_survey_end_before_start()`, `check_duration_by_hh_size()`, and `check_roster_completeness()` are also classified as logic consistency checks. Their documentation appears in Sections 5.6 and 5.3, respectively.

5.10 Text Quality Checks

These functions validate open-ended text responses, names, phone numbers, and “other (specify)” fields.

5.10.1 `check_other_specify()`

Signature:

```
check_other_specify(data, id_col, other_col)
```

Description: Flags `other (specify)''` responses that are empty, too short, or contain placeholder text (e.g., `xxx'`, `test''`, `asdf''`). Severity: warning.

Example:

```
res <- check_other_specify(  
  survey, id_col = "hh_id", other_col = "crop_other"  
)
```

5.10.2 `check_text_length()`

Signature:

```
check_text_length(data, id_col, text_col,  
  min_length = 3, max_length = 500)
```

Description: Flags text responses shorter than `min_length` or longer than `max_length` characters. Severity: warning.

Example:

```
res <- check_text_length(  
  survey, id_col = "hh_id", text_col = "comments",  
  min_length = 3, max_length = 500  
)
```

5.10.3 check_text_duplicates()

Signature:

```
check_text_duplicates(data, id_col, text_col, min_length = 10)
```

Description: Identifies duplicate text responses (after trimming and case-folding) that are at least `min_length` characters long. Identical long-form responses across different records may indicate copy-paste fabrication. Severity: **warning**.

Example:

```
res <- check_text_duplicates(  
  survey, id_col = "hh_id", text_col = "comments",  
  min_length = 10  
)
```

5.10.4 check_name_format()

Signature:

```
check_name_format(data, id_col, name_col)
```

Description: Validates name fields for common issues: all-caps, numeric characters, single characters, or known test entries. Severity: **info**.

Example:

```
res <- check_name_format(  
  survey, id_col = "hh_id", name_col = "head_name"  
)
```

5.10.5 check_phone_format()

Signature:

```
check_phone_format(data, id_col, phone_col, country_code,  
  min_digits = 8, max_digits = 15)
```

Description: Validates phone numbers against country-specific formatting rules: correct number of digits, valid prefix, and no invalid characters. Severity: **warning**.

Example:

```
res <- check_phone_format(  
  survey, id_col = "hh_id", phone_col = "phone",  
  country_code = "BF", min_digits = 8, max_digits = 8  
)
```

5.11 Back-Check Checks

Back-checks (re-interviews of a subsample) are a cornerstone of survey quality assurance. These functions compare original and back-check data.

5.11.1 check_backcheck_t1_match()

Signature:

```
check_backcheck_t1_match(data, id_col, backcheck_data, bc_id_col,  
  t1_cols, ...)
```

Description: Type~1 back-check: compares Type~1 variables (e.g., gender, location) between original and back-check data. Any mismatch on these variables signals a serious data quality issue because they should never change. Severity: **error**.

Example:

```
res <- check_backcheck_t1_match(  
  data = survey, id_col = "hh_id",  
  backcheck_data = bc_data, bc_id_col = "hh_id",  
  t1_cols = c("gender", "village")  
)
```

5.11.2 check_backcheck_t2_match()

Signature:

```
check_backcheck_t2_match(data, id_col, backcheck_data, bc_id_col,  
  t2_cols, max_error_rate = 0.10,  
  enum_col = NULL, ...)
```

Description: Type~2 back-check: compares categorical or yes/no responses between original and back-check. Computes exact-match agreement rates per variable and flags enumerators whose error rate exceeds `max_error_rate`. Severity: **warning**.

Example:

```
res <- check_backcheck_t2_match(  
  data = survey, id_col = "hh_id",  
  backcheck_data = bc_data, bc_id_col = "hh_id",  
  t2_cols = c("roof_material", "wall_material", "water_source"),  
  max_error_rate = 0.10, enum_col = "enum_id"  
)
```

5.11.3 check_backcheck_t3_match()

Signature:

```
check_backcheck_t3_match(data, id_col, backcheck_data, bc_id_col,  
  t3_cols, ...)
```

Description: Type~3 back-check: compares variables that are expected to change between original and back-check (e.g., satisfaction, attitude). This check is purely informational and computes change rates per variable. Severity: **info**.

Example:

```
res <- check_backcheck_t3_match(  
  data = survey, id_col = "hh_id",  
  backcheck_data = bc_data, bc_id_col = "hh_id",  
  t3_cols = c("satisfaction", "attitude")  
)
```

5.11.4 check_backcheck_coverage()

Signature:

```
check_backcheck_coverage(data, id_col, backcheck_data, bc_id_col,  
  strata_col = NULL, min_coverage = 0.10,  
  ...)
```

Description: Verifies that a sufficient percentage of original surveys have been back-checked. If strata are provided, coverage is computed per stratum. Severity: **warning**.

Example:

```
res <- check_backcheck_coverage(  
  data = survey, id_col = "hh_id",  
  backcheck_data = bc_data, bc_id_col = "hh_id",  
  strata_col = "region", min_coverage = 0.10  
)
```

5.11.5 check_backcheck_by_enumerator()

Signature:

```
check_backcheck_by_enumerator(data, id_col, enum_col,  
                              backcheck_data, bc_id_col,  
                              compare_cols, threshold_sd = 2,  
                              ...)
```

Description: Computes per-enumerator error rates across all comparison columns and flags enumerators whose error rate is more than `threshold_sd` standard deviations above the team mean. Severity: `warning`.

Example:

```
res <- check_backcheck_by_enumerator(  
  data = survey, id_col = "hh_id", enum_col = "enum_id",  
  backcheck_data = bc_data, bc_id_col = "hh_id",  
  compare_cols = c("roof_material", "hh_size"),  
  threshold_sd = 2  
)
```

5.12 Paradata Checks

Paradata (text audit, server logs, device metadata) provides fine-grained quality signals at the question level.

5.12.1 check_text_audit_duration()

Signature:

```
check_text_audit_duration(text_audit, id_col, question_col,  
                          duration_col, min_seconds = 1,  
                          max_seconds = 300)
```

Description: Flags individual question responses with implausible durations (too fast or too slow) from text audit paradata. Severity: **warning**.

Example:

```
res <- check_text_audit_duration(  
  text_audit, id_col = "hh_id",  
  question_col = "question", duration_col = "duration_sec",  
  min_seconds = 1, max_seconds = 300  
)
```

5.12.2 check_text_audit_sequence()

Signature:

```
check_text_audit_sequence(text_audit, id_col, question_col,  
                          expected_order)
```

Description: Verifies that questions were answered in the expected order. Out-of-order responses may indicate form manipulation. Severity: **info**.

Example:

```
expected <- c("consent", "hh_size", "income", "expenditure")  
res <- check_text_audit_sequence(  
  text_audit, id_col = "hh_id",  
  question_col = "question", expected_order = expected  
)
```

5.12.3 check_speed_violations()

Signature:

```
check_speed_violations(text_audit, id_col, question_col,  
                       duration_col, min_seconds_per_field = 0.5)
```

Description: Counts the number of fields answered faster than `min_seconds_per_field`. Interviews with many speed violations are flagged. Severity: **warning**.

Example:

```
res <- check_speed_violations(  
  text_audit, id_col = "hh_id",  
  question_col = "question", duration_col = "duration_sec",  
  min_seconds_per_field = 0.5  
)
```

5.12.4 check_field_comments()

Signature:

```
check_field_comments(data, id_col, comment_col)
```

Description: Reports observations that have non-empty field-level comments, which may contain corrections, clarifications, or flags left by enumerators. Severity: **info**.

Example:

```
res <- check_field_comments(  
  survey, id_col = "hh_id", comment_col = "enumerator_notes"  
)
```

5.12.5 check_photo_attachment()

Signature:

```
check_photo_attachment(data, id_col, photo_col)
```

Description: Flags records where a required photo attachment is missing or has an invalid path. Severity: **warning**.

Example:

```
res <- check_photo_attachment(  
  survey, id_col = "hh_id", photo_col = "dwelling_photo"  
)
```

5.12.6 check_device_consistency()

Signature:

```
check_device_consistency(data, id_col, enum_col, device_col)
```

Description: Flags enumerators who used more than one device during data collection. While not always problematic, device switching may warrant investigation. Severity: **info**.

Example:

```
res <- check_device_consistency(  
  survey, id_col = "hh_id",  
  enum_col = "enum_id", device_col = "device_id"  
)
```

5.13 Correction Functions

These functions apply corrections to the dataset based on a correction log and recode open-ended responses.

5.13.1 apply_corrections()

Signature:

```
apply_corrections(data, correction_log, id_col, variable_col,  
  old_value_col, new_value_col)
```

Description: Applies a correction log to the dataset. For each row in the log, the function locates the matching record by `id_col` and variable name, verifies the current value matches `old_value_col`, and replaces it with `new_value_col`. Returns the corrected dataset along with a log of applied and failed corrections.

Example:

```
corrections <- data.frame(  
  hh_id      = c("HH-0010", "HH-0015"),  
  variable   = c("duration_min", "income"),  
  old_value  = c("3", "1000000000"),  
  new_value  = c("30", "500000"),  
  stringsAsFactors = FALSE  
)
```

```
corrected <- apply_corrections(  
  survey, correction_log = corrections,  
  id_col = "hh_id", variable_col = "variable",  
  old_value_col = "old_value", new_value_col = "new_value"  
)
```

5.13.2 check_correction_log()

Signature:

```
check_correction_log(data, correction_log, id_col, variable_col,  
  old_value_col)
```

Description: Validates a correction log before applying it. Checks that all referenced IDs exist, variable names are valid, and old values match current data. Returns a `check_result` with mismatches flagged. Severity: **error**.

Example:

```
res <- check_correction_log(  
  survey, correction_log = corrections,  
  id_col = "hh_id", variable_col = "variable",  
  old_value_col = "old_value"  
)
```

5.13.3 recode_other_specify()

Signature:

```
recode_other_specify(data, id_col, other_col, recode_map,  
  target_col = NULL)
```

Description: Recodes free-text “other (specify)” responses into existing categories using a named mapping vector. Optionally writes recoded values into `target_col` (the main categorical variable). Returns the recoded dataset.

Example:

```
recode_map <- c(
  "millet"      = "cereals",
  "sorghum"     = "cereals",
  "groundnut"  = "legumes"
)

recoded <- recode_other_specify(
  survey, id_col = "hh_id",
  other_col = "crop_other", recode_map = recode_map,
  target_col = "main_crop"
)
```

6 Configuration System

The YAML configuration system allows survey managers to define all check parameters in a single file, enabling reproducible and auditable quality assurance.

6.1 YAML Structure

A configuration file has three top-level sections:

```
# config.yml
project:
  name: "Household Survey 2025"
  survey_start: "2025-01-15"
  survey_end: "2025-06-30"

variables:
  id: "hh_id"
  enumerator: "enum_id"
  date: "interview_date"
  duration: "duration_min"
  gps_latitude: "gps_lat"
  gps_longitude: "gps_lon"
  gps_accuracy: "gps_accuracy"
  consent: "consent"
  form_version: "form_version"
  hh_size: "hh_size"

checks:
  identification:
    duplicate_ids: true
    missing_ids: true
    id_format:
      pattern: "^HH-[0-9]{4}$"

  metadata:
    consent:
      consent_value: 1
    form_version:
      expected_version: "v2.1"

  completeness:
    missing_by_variable:
      threshold: 0.05
    missing_by_enumerator:
      threshold_sd: 2

outliers:
```

```
outliers_iqr:
  columns: ["income", "expenditure", "hh_size"]
  multiplier: 1.5
hard_range:
  - column: "hh_size"
    min_val: 1
    max_val: 30
  - column: "income"
    min_val: 0
    max_val: 50000000

timing:
  survey_duration:
    min_duration: 15
    max_duration: 120
  collection_window:
    start_date: "2025-06-01"
    end_date: "2025-06-30"

gps:
  gps_boundary:
    boundary_bbox: [-1.7, 12.1, -1.2, 12.6]
  gps_null_island: true
  gps_accuracy:
    max_accuracy: 50

enumerator:
  enumerator_productivity:
    max_daily: 10
  enumerator_duration:
    threshold_sd: 2

fabrication:
  benford_first_digit:
    columns: ["income", "expenditure"]
  icc_enumerator:
    num_cols: ["income", "expenditure", "hh_size"]
    max_icc: 0.15
```

6.2 Configuration Functions

```
# Read and validate a YAML config file
config <- read_config("config.yml")

# Validate config structure
validate_config(config)
```

```
#> Config validation passed

# Extract a nested value with a default
id_col <- cfg_get(config, "variables", "id")
max_daily <- cfg_get(config, "checks", "enumerator",
  "productivity", "max_daily",
  default = 10)
```

7 Orchestrator: `run_all_checks()`

The orchestrator is the primary entry point for running a complete suite of checks in a single call.

7.1 Signature

```
run_all_checks(  
  data,  
  config,  
  sampling_frame = NULL,  
  backcheck_data = NULL,  
  verbose        = TRUE,  
  ...  
)
```

7.2 Parameters

Parameter	Description
<code>data</code>	Main survey data frame
<code>config</code>	Either a path to a YAML config file or an <code>adc_config</code> object
<code>sampling_frame</code>	Optional character vector of valid IDs
<code>backcheck_data</code>	Optional back-check data frame
<code>verbose</code>	If <code>TRUE</code> , prints progress messages
<code>...</code>	Reserved for future use

7.3 Return Value

Returns an object of class `adc_report`, which is a named list containing:

- `results`: A list of `check_result` objects
- `config`: The configuration used
- `metadata`: Timestamp, package version, data dimensions
- `summary`: Aggregated counts by severity and category

7.4 Methods

```
report <- run_all_checks(survey, config = config, verbose = TRUE)

# Print overview
print(report)
#> === afcestDataCheck Report ===
#> Data: 200 observations, 12 variables
#> Checks run: 18
#> Errors: 3 | Warnings: 12 | Info: 2

# Detailed summary
summary(report)
#> Category           Checks  Errors  Warnings  Info
#> identification      3        1         0        0
#> metadata             2        1         0        0
#> completeness        2        0         2        0
#> outliers             3        0         3        0
#> ...
```


8 Platform Normalization

The package supports data from five major survey platforms by normalizing platform-specific conventions into a common format.

8.1 Supported Platforms

Table 2: Supported data collection platforms

Platform	GPS Format	Duration	Metadata Cols
KoBoToolbox	String: "lat lon alt acc"	Seconds (integer)	<code>_submission_time</code>
SurveyCTO	String: "lat lon alt acc"	Seconds (integer)	<code>starttime</code> , <code>endtime</code>
ODK Central	GeoJSON or string	Seconds	<code>__system/*</code>
Survey Solutions	Separate lat/lon columns	Minutes (decimal)	<code>interview_*</code>
CSPPro	Separate lat/lon columns	Varies	<code>CAPI_*</code>

8.2 Key Functions

```
# Auto-detect platform and normalize
normalized <- normalize_survey_data(
  raw_data, platform = "auto"
)
# Returns data with standardized column names:
#   .id, .enum_id, .start_time, .end_time, .duration_min,
#   .lat, .lon, .gps_accuracy, .submission_date

# Parse a GPS string (KoBoToolbox / SurveyCTO format)
coords <- parse_gps_string("12.345 -1.234 300 5.2")
coords
#> $lat
#> [1] 12.345
#> $lon
#> [1] -1.234
#> $alt
#> [1] 300
#> $accuracy
#> [1] 5.2

# Detect platform from column names (internal)
# detect_platform(data) -> "kobo" | "scto" | "odk" | "suso" | "cspro"
```

9 Export Functions

9.1 Excel Export

```
export_to_excel(report$results, data = survey, id_col = "hh_id",  
               path = "hfc_report.xlsx")
```

The Excel workbook contains:

1. **Summary** sheet: Overview with counts by category and severity, colored by severity level (red = error, orange = warning, blue = info).
2. **Per-check sheets**: One sheet per check with flagged IDs, reasons, and relevant data columns. Conditional formatting highlights flagged cells.
3. **Enumerator Summary**: Aggregated error/warning counts per enumerator, sorted by total flags (descending).

9.2 CSV Export

```
export_to_csv(report$results, output_dir = "hfc_output/")
```

Generates two files in `dir`:

- `hfc_summary.csv`: One row per check with name, category, severity, and flag count.
- `hfc_flagged.csv`: Long-format table of all flagged observations with ID, check name, reason, and severity.

10 Integration with Shiny

The standardized `check_result` object integrates naturally with Shiny reactive programming. Below is a minimal example.

```
library(shiny)
library(bslib)
library(afcestDataCheck)

ui <- page_sidebar(
  title = "HFC Dashboard",
  sidebar = sidebar(
    fileInput("upload", "Upload survey data (.csv)",
      actionButton("run", "Run checks", class = "btn-primary")
    ),
    card(
      card_header("Results"),
      tableOutput("summary_table")
    ),
    card(
      card_header("Flagged Observations"),
      tableOutput("flagged_table")
    )
  )
)

server <- function(input, output, session) {
  survey_data <- reactive({
    req(input$upload)
    read.csv(input$upload$datapath)
  })

  report <- eventReactive(input$run, {
    config <- read_config("config.yml")
    run_all_checks(survey_data(), config = config, verbose = FALSE)
  })

  output$summary_table <- renderTable({
    req(report())
    summary(report())
  })

  output$flagged_table <- renderTable({
    req(report())
    results <- report()$results
    flagged <- bind_check_results(results)
    flagged[flagged$n_flagged > 0, ]
  })
}
```

```
}  
  
shinyApp(ui, server)
```

11 Appendix: Complete Function Index

Table 3: Complete function index for **afcestDataCheck** v0.3.0

#	Function	Category	Severity	Section
1	check_duplicate_ids	Identification	error	5.1
2	check_duplicate_fingerprint	Identification	warning	5.1
3	check_missing_ids	Identification	error	5.1
4	check_id_format	Identification	warning	5.1
5	check_id_in_sample	Identification	warning	5.1
6	check_consent	Metadata	error	5.2
7	check_form_version	Metadata	warning	5.2
8	check_interview_completed	Metadata	warning	5.2
9	check_survey_tracking	Metadata	info	5.2
10	check_respondent_eligibility	Metadata	warning	5.2
11	check_missing_by_variable	Completeness	warning	5.3
12	check_missing_by_enumerator	Completeness	warning	5.3
13	check_all_missing_variables	Completeness	error	5.3
14	check_skip_pattern	Completeness	warning	5.3
15	check_roster_completeness	Completeness	warning	5.3
16	check_outliers_iqr	Outliers	warning	5.4
17	check_outliers_zscore	Outliers	warning	5.4
18	check_outliers_mad	Outliers	warning	5.4
19	check_hard_range	Outliers	error	5.4
20	check_negative_values	Outliers	error	5.4
21	check_enumerator_productivity	Enumerator	warning	5.5
22	check_enumerator_duration	Enumerator	warning	5.5
23	check_dk_rate	Enumerator	warning	5.5
24	check_enumerator_time_gap	Enumerator	warning	5.5
25	check_straightlining	Enumerator	warning	5.5
26	check_survey_duration	Timing	warning	5.6
27	check_collection_window	Timing	warning	5.6
28	check_future_dates	Timing	error	5.6
29	check_survey_end_before_start	Timing	error	5.6
30	check_duration_by_hh_size	Timing	warning	5.6
31	check_gps_boundary	GPS	error	5.7
32	check_gps_centroid_distance	GPS	warning	5.7
33	check_gps_accuracy	GPS	warning	5.7
34	check_gps_duplicates	GPS	warning	5.7
35	check_gps_swap	GPS	error	5.7
36	check_gps_null_island	GPS	error	5.7
37	check_gps_clustering	GPS	warning	5.7
38	check_gps_altitude	GPS	warning	5.7
39	check_benford_first_digit	Fabrication	info	5.8
40	check_benford_second_digit	Fabrication	info	5.8
41	check_digit_preference	Fabrication	info	5.8
42	check_icc_enumerator	Fabrication	warning	5.8
43	check_response_entropy	Fabrication	warning	5.8
44	check_duplicate_response_patterns	Fabrication	warning	5.8

#	Function	Category	Severity	Section
45	check_variance_ratio	Fabrication	warning	5.8
46	check_hh_composition	Logic	warning	5.9
47	check_income_expenditure	Logic	warning	5.9
48	check_age_date_consistency	Logic	warning	5.9
49	check_custom_logic	Logic	warning	5.9
50	check_other_specify	Text	warning	5.10
51	check_text_length	Text	warning	5.10
52	check_text_duplicates	Text	warning	5.10
53	check_name_format	Text	info	5.10
54	check_phone_format	Text	warning	5.10
55	check_backcheck_t1_match	Back-Check	error	5.11
56	check_backcheck_t2_match	Back-Check	warning	5.11
57	check_backcheck_t3_match	Back-Check	info	5.11
58	check_backcheck_coverage	Back-Check	warning	5.11
59	check_backcheck_by_enumerator	Back-Check	warning	5.11
60	check_text_audit_duration	Paradata	warning	5.12
61	check_text_audit_sequence	Paradata	info	5.12
62	check_speed_violations	Paradata	warning	5.12
63	check_field_comments	Paradata	info	5.12
64	check_photo_attachment	Paradata	warning	5.12
65	check_device_consistency	Paradata	info	5.12
66	apply_corrections	Corrections	—	5.13
67	check_correction_log	Corrections	error	5.13
68	recode_other_specify	Corrections	—	5.13
69	new_check_result	Infrastructure	—	4
70	is_check_result	Infrastructure	—	4
71	bind_check_results	Infrastructure	—	4
72	read_config	Configuration	—	6
73	validate_config	Configuration	—	6
74	cfg_get	Configuration	—	6
75	run_all_checks	Orchestrator	—	8
76	normalize_survey_data	Platform	—	9
77	parse_gps_string	Platform	—	9
78	export_to_excel	Export	—	10
79	export_to_csv	Export	—	10

afcestDataCheck is developed and maintained by
AfCEST — African Center for Studies and Training
 Ouagadougou, Burkina Faso

<https://github.com/afcest/afcestDataCheck>